Part 4 — Advanced SQL Problems (5 Queries)

A1 — Running Total using Window Functions

Concept: Window functions compute values across a set of rows related to the current row without collapsing them. SUM() OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) gives a running cumulative total.

SQL:

```
SELECT id, jobcat, salary,
       SUM(salary) OVER (
           PARTITION BY jobcat
           ORDER BY id
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total_salary
FROM employees ORDER BY jobcat, id;
```

Pandas:

```
df.sort_values(['jobcat','id'])
df['running_total'] = df.groupby('jobcat')['salary'].cumsum()
```

Key concept: PARTITION BY = groupby, ORDER BY = sort within group, cumsum() = running total.

A2 — Top 3 Per Category using RANK()

Concept: RANK() assigns a rank number within a partition. Unlike ROW_NUMBER(), RANK() gives the same rank to ties and skips subsequent ranks.

SQL:

```
SELECT * FROM (
    SELECT id, gender, jobcat, salary,
           RANK() OVER (PARTITION BY jobcat ORDER BY salary DESC) AS salary_rank
    FROM employees
) AS ranked WHERE salary_rank <= 3;
```

Pandas:

```
df['salary_rank'] = df.groupby('jobcat')['salary']
                      .rank(method='min', ascending=False).astype(int)
result = df[df['salary_rank'] <= 3]
```

method='min' replicates SQL RANK() behavior — ties share the lowest rank. method='dense' would replicate DENSE_RANK().

A3 — Month-over-Month Growth using LAG()

Concept: LAG(col, n) fetches the value from n rows behind in the current window. Used to compare current vs previous period. NULLIF prevents division-by-zero.

SQL:

```sql
SELECT id, jobcat, salary,
       LAG(salary) OVER (PARTITION BY jobcat ORDER BY id) AS prev_salary,
       salary - LAG(salary) OVER (...) AS salary_diff,
       ROUND((salary - LAG(salary) OVER (...)) * 100.0
       / NULLIF(LAG(salary) OVER (...), 0), 2) AS pct_change
FROM employees ORDER BY jobcat, id;
```

Pandas:

```python
df['prev_salary'] = df.groupby('jobcat')['salary'].shift(1)
df['salary_diff'] = df['salary'] - df['prev_salary']
df['pct_change'] = (df['salary_diff'] / df['prev_salary'] * 100).round(2)
```

shift(1) = LAG(1). shift(-1) = LEAD(1). NULLIF(...,0) = handled automatically since division by NaN returns NaN.

A4 — CTE for Multi-Step Calculation

Concept: Common Table Expressions (CTEs) using WITH clauses let you break a complex query into named intermediate steps, improving readability and reuse.

SQL:

```sql
WITH dept_avg AS (
    SELECT jobcat, AVG(salary) AS avg_salary
    FROM employees GROUP BY jobcat
),
salary_growth AS (
    SELECT id, gender, jobcat, salary, salbegin,
           ROUND((salary - salbegin)*1.0/salbegin*100, 2) AS growth_pct
    FROM employees
)
SELECT sg.*, da.avg_salary,
       CASE WHEN sg.salary > da.avg_salary THEN 'Above Avg' ELSE 'Below Avg' END
FROM salary_growth sg JOIN dept_avg da ON sg.jobcat = da.jobcat
ORDER BY growth_pct DESC;
```

Pandas:

```python
# CTE 1 — dept_avg
dept_avg = df.groupby('jobcat')['salary'].mean().reset_index()
# CTE 2 — salary_growth
sg = df.copy()
sg['growth_pct'] = ((sg['salary'] - sg['salbegin']) / sg['salbegin'] *
100).round(2)
# Final join + label
result = sg.merge(dept_avg, on='jobcat')
result['status'] = result.apply(lambda r: 'Above Avg' if r['salary'] >
r['avg_salary'] else 'Below Avg', axis=1)
```

Each Pandas intermediate DataFrame = one CTE. Chaining .merge() calls = multiple CTEs joined together.

A5 — Correlated Subquery

Concept: A correlated subquery runs once per outer row and references the outer query. It cannot be evaluated independently — it depends on the current row's value (here: e.jobcat).

SQL:

```
SELECT id, gender, jobcat, salary
FROM employees e
WHERE salary > (
    SELECT AVG(salary) FROM employees
    WHERE jobcat = e.jobcat  -- References outer row
)
ORDER BY jobcat, salary DESC;
```

Pandas:

```
dept_avg = df.groupby('jobcat')['salary'].mean().reset_index()
merged = df.merge(dept_avg, on='jobcat', suffixes=('','_dept'))
result = merged[merged['salary'] > merged['salary_dept']]
```

Pandas cannot natively run correlated subqueries row-by-row efficiently. The standard approach is to pre-compute the correlated value (groupby aggregate), merge it back, then filter — same result, far better performance.

Interview Prep — Key Concepts

Concept	SQL Syntax	Pandas Equivalent	When to Use
Filter rows	WHERE col = val	df[df['col'] == val]	Basic row selection
Multi-condition filter	WHERE A AND B	df[(A) & (B)]	Combining conditions
Aggregation	GROUP BY + COUNT/AVG/SUM	groupby().agg()	Summary statistics per group
Self-join	FROM t a JOIN t b ON ...	df.merge(df, on=..., suffixes)	Compare rows within same table
Running total	SUM() OVER (ORDER BY)	.groupby().cumsum()	Cumulative metrics
Rank within group	RANK() OVER (PARTITION BY)	.groupby().rank(method='min')	Top-N per category
Previous row value	LAG(col) OVER (PARTITION BY ... ORDER BY)	.groupby().shift(1)	Period-over-period comparison

Concept	SQL Syntax	Pandas Equivalent	When to Use
Intermediate steps	WITH cte AS (...)	Intermediate DataFrames	Multi-step complex queries
Correlated subquery	WHERE col > (SELECT ... WHERE x = e.x)	groupby merge + filter	Row-level comparison to group stat